

AI-Powered Resume Screening System

Resume Analyzer — Advanced ATS

A hybrid BERT and rule-based Applicant Tracking System that screens and ranks resumes by meaning, not just keywords.

 [Python](#)

 [Streamlit](#)

 [Sentence-Transformers](#)

 [License](#)

 [Status](#)

[Overview](#) · [Architecture](#) · [Screening Pipeline](#) ·
[How Scoring Works](#) · [Key Features](#) · [Tech Stack](#) ·
[Getting Started](#) · [Testing](#)

Overview

Traditional Applicant Tracking Systems (ATS) rank resumes using exact keyword matching. A

resume that says "Artificial Intelligence" instead of "AI" can be unfairly rejected, even if the candidate is fully qualified.

AI-Powered Resume Screening System

addresses this by combining:

- BERT-based semantic similarity (`all-MiniLM-L6-v2`) to understand meaning, not just words
- Rule-based skill extraction with synonym resolution across a 9-category, 100+ skill database
- Automated experience and education parsing from unstructured resume text
- A transparent 4-component weighted scoring formula that produces a fair, explainable ATS score

The result is a multi-tenant recruiting platform, not a script — with separate recruiter and candidate experiences, a hiring pipeline, and bulk-screening support for hundreds of resumes at once.

Architecture

System Architecture

The application is organized into four layers: user roles (recruiter, candidate, host/admin), the Streamlit application layer, the AI core engine, and a multi-tenant data layer.

Screening Pipeline

Screening Pipeline

End-to-end flow from upload to ranked results: job description and resume upload, text extraction, NLP preprocessing, parallel BERT semantic encoding and rule-based skill/experience/education extraction, hybrid scoring, and final ranked output.

How Scoring Works

Scoring Weights

Each resume receives a single ATS score (0–100) built from four weighted components:

Component	Weight	What it measures
BERT Semantic Similarity	40%	Contextual meaning match between resume and job description using Sentence-Transformers embeddings, rescaled to remove score-ceiling bias
Rule-Based Skill Match	35%	Curated 9-category, 100+ skill database with synonym resolution (for example, ML to Machine Learning, K8s to Kubernetes)
Education Relevance	15%	Regex-based degree and qualification detection compared against job requirements
Experience Match	10%	Years-of-experience extraction from

Component	Weight	What it measures
		unstructured resume text

Key Features

- Hybrid semantic and rule-based engine that goes beyond keyword matching to understand context
- Multi-tenant architecture with isolated recruiter workspaces, work-email domain verification, and a host approval queue
- Recruiter Kanban pipeline with hiring stages, job postings, interview scheduling, and recruiter notes
- Candidate portal — a public-facing career site where candidates can apply directly
- Bulk screening at scale, validated against 300 synthetic resumes for large-batch processing
- Audit trail logging every recruiter action for accountability
- One-click CSV export of ranked results for offline review

- 59-check automated test suite regression-testing scoring logic before every release
-

Tech Stack

Layer	Technology
Language	Python 3.10+
Web App / UI	Streamlit
Semantic AI	Sentence-Transformers (all-MiniLM-L6-v2)
NLP	NLTK, regex-based rule engine
Resume Parsing	pdfplumber
ML Utilities	scikit-learn, NumPy, pandas
Testing	Pytest (59-check regression suite)

Project Structure

```
ai-resume-screening-system/
├─ app.py #
Streamlit entry point
├─ ai_engine.py #
Core hybrid scoring engine (BERT +
rule-based)
├─ test_ai_engine.py #
59-check automated test suite
├─ modules/
│   └─ parser.py #
Resume text extraction (pdfplumber)
│   └─ skill_extractor.py #
Rule-based skill and synonym resolution
│   └─ semantic_matcher.py #
BERT embedding and similarity scoring
│   └─ scoring.py #
4-component weighted ATS scoring
├─ portals/
│   └─ recruiter_dashboard.py #
Kanban pipeline, job postings, audit
trail
│   └─ candidate_portal.py #
Public career site and applications
├─ assets/
│   └─ architecture.png #
System architecture diagram
│   └─ pipeline.png #
Screening pipeline diagram
│   └─ scoring.png #
Scoring weights diagram
```

```
├── data/
│   └── synthetic_resumes/          #
300 synthetic resumes used for bulk
testing
├── requirements.txt
└── README.md
```

Getting Started

Prerequisites

- Python 3.10 or higher
- pip

Installation

```
git clone https://github.com/<your-
username>/ai-resume-screening-
system.git
cd ai-resume-screening-system
pip install -r requirements.txt
```

Run the app

```
python -m streamlit run app.py
```

The app opens in your browser at
`http://localhost:8501` .

Testing and Validation

The scoring engine is backed by a 59-check automated test suite covering skill extraction accuracy, semantic score calibration, and edge-case handling (for example, false-positive matches like "cv" or ambiguous abbreviations). All bulk-screening flows were validated against 300 synthetic resumes before release.

```
pytest test_ai_engine.py -v
```

Module	Pass Rate
Skill Extraction	96%
BERT Semantic Matching	94%
Experience Parsing	98%
Education Parsing	97%
Synonym Resolution	96%

Roadmap

- Resume-to-role recommendation engine
- Multi-language resume support
- Interview scheduling integrations (Google Calendar, Outlook)
- Explainability panel showing why a resume received its score

License

This project is licensed under the MIT License.
See [LICENSE](#) for details.

Author

S Mohammed Salman

B.Tech Artificial Intelligence and Data Science,
RVS Technical Campus, Coimbatore
Built during an internship at Sri Lakshmi
Technology